Instagram reel import

Implementation spec for NQ-Shoppable v2

Importing a merchant's own Instagram reels into R2 by connecting their Instagram Business or Creator account through Meta's OAuth, so that ownership of every imported reel is proved by an access token rather than asserted by whoever pasted the link.

Scope	Merchant-owned reels only. No scraping, no third-party downloader services, no private or unauthorised accounts.
Depends on	Meta app with Instagram product, app review for `instagram_business_basic`, business verification.
Touches	New routes, new Prisma model, one new server module. Existing ffmpeg and R2 pipeline reused unchanged.
Build estimate	4–6 engineering days, plus 2–4 weeks of Meta app review running in parallel.
Status	Design approved in principle. Not yet implemented.

1. Summary

Merchants want to add videos to their store by pasting an Instagram reel link instead of exporting the file and uploading it. The hard part is not fetching the video — it is proving the merchant is entitled to that video before we re-host it on a client storefront.

The decision

Do not verify the link. Verify the account, once, with OAuth, and then use the link purely as a lookup key.

An Instagram shortcode carries no owner information. `instagram.com/reel/DQx7abcDEF/` tells us nothing about who posted it; the only way to find out from the URL alone is to fetch the page, which is the scraping route we have already ruled out on ToS, copyright and operational-fragility grounds.

Instead the merchant connects their Instagram Business or Creator account through Meta's own OAuth. We store a long-lived token against the shop. From that point `GET graph.instagram.com/me/media` returns that account's media and nothing else. Ownership is a property of the token, not a claim in a form field. Private accounts, competitors' accounts and unauthorised accounts all fail identically — they were never in the response.

What the merchant sees

Step	Screen	What happens
1	Settings → Instagram	“Connect Instagram” button. One OAuth round trip. Shows **@handle connected** afterwards.
2	Videos → Import from Instagram	Grid of their own reels with thumbnails, captions and dates. Click to select.
3	Same screen	Tag a product, as with a normal upload. Import runs in the background.
4	Videos list	The reel appears with a processing bar, exactly like a file upload.

The paste-a-link box stays, as a secondary path: paste a link, we match its shortcode against the connected account's permalinks, and import if it matches. If it does not match, we reject it with a specific message rather than attempting a fetch.

The one constraint that shapes everything

> **Not every reel is downloadable**
>
> Meta omits `media_url` for any video whose audio is copyrighted or licensed — including tracks added from Instagram's own audio library, and anything flagged for a copyright violation. A large share of real merchant reels fall into this bucket. They will be listed and visible in the picker but cannot be imported.
>
> This is inconvenient but not perverse: those are exactly the reels that would expose a client store to a music licensing claim if we re-hosted them. Meta's restriction and our own legal exposure point the same way. Section 8 covers the UX for it.

2. Alternatives considered

Recorded so the decision is not relitigated later.

Approach	How it verifies	Verdict
Scrape the reel page	Fetch the public page, read the owner handle, compare to a handle the merchant typed.	**Rejected.** Breaches Instagram's terms, the handle field is merchant-supplied and therefore not evidence, and Instagram rate-limits and IP-blocks server ranges. We already hit 429s on this path.
Third-party downloader API	None. Pays a service to scrape on our behalf.	**Rejected.** Moves the ToS breach one hop away without removing it, and puts an unknown third party in the critical path of a paid client feature.
Bio-code challenge	Merchant puts a code such as `NQ-7F2A` in their Instagram bio; we read it back.	**Rejected.** Reading the bio still means fetching the profile, so it inherits the scraping problem while adding merchant friction.
Merchant attestation only	Checkbox: “I own this content or have rights to use it”, stored with shop, handle, timestamp and IP.	**Kept as a supplement.** A liability shield, not verification — it stops nothing technically. Worth logging alongside OAuth, not instead of it.
Manual file upload	Merchant saves the reel from the Instagram app and uploads the file.	**Kept as the fallback.** Zero API surface, zero ToS risk, worse UX. This is what the flow degrades to when `media_url` is missing.
Instagram Login OAuth	Merchant authorises our Meta app. We read only their own media, using a token they granted.	**Chosen.** The only option where ownership is actually proved, and the only sanctioned download path.

3. Architecture

Two flows. Connect happens once per shop; import happens per reel.

Connect (once per shop)

```
Merchant                 NQ app (Fly)                    Meta
   |                          |                            |
   |-- click Connect -------->|                            |
   |                          |-- 302 to /oauth/authorize ->|
   |<------------- Instagram consent screen ----------------|
   |-- approve ------------------------------------------->|
   |                          |<-- 302 ?code=... ----------|
   |                          |-- POST /oauth/access_token ->|   short-lived, 1h
   |                          |<-- access_token, user_id ---|
   |                          |-- GET /access_token --------->|  ig_exchange_token
   |                          |<-- long-lived token, 60d ---|
   |                          |-- upsert InstagramAccount    |
   |<-- "@handle connected" --|                            |
```

Import (per reel)

```
  pick reel in UI  (or paste link -> match shortcode against permalinks)
        |
        v
  GET graph.instagram.com/{media-id}?fields=media_url,thumbnail_url,permalink,caption,...
        |
        +-- media_url absent  ->  reject: "licensed audio, upload the file instead"
        |
        v
  stream media_url -> Buffer -> uploadToR2("videos/{ts}-ig-{shortcode}.mp4")
        |
        v
  db.video.create({ ...product tagging, sourceIgShortcode, sourcePermalink })
        |
        v
  processVideo(videoUrl, baseKey, onProgress)      <-- existing pipeline, untouched
        |
        v
  pushVideoToProduct(...) if a product was tagged  <-- existing auto-add, untouched
```

Everything below the `uploadToR2` line is the code path we already run for file uploads in `app/routes/app.videos.new.tsx`. The Instagram work is entirely upstream of it: fetch bytes, hand them to R2, then join the existing flow. That is deliberate — compression, thumbnailing, the short-video path, progress reporting, retry and the Shopify product-media auto-add all keep working with no changes.

New files

Path	Purpose
`app/instagram.server.ts`	All Meta API calls: OAuth exchange, token refresh, list media, fetch one media, shortcode matching.
`app/routes/app.instagram.tsx`	Settings screen. Connect / disconnect, shows connected handle and token expiry.
`app/routes/auth.instagram.callback.tsx`	OAuth redirect target. Exchanges code, stores token, redirects back into the embedded app.
`app/routes/app.videos.instagram.tsx`	Reel picker grid plus the import action.
`app/routes/api.instagram.refresh.tsx`	Token refresh endpoint, hit by a Fly scheduled machine or cron.
`prisma/migrations/…_add_instagram_account/`	New model plus two columns on Video.

4. Meta app setup

Done once, by us, not per client.

4.1 One shared Meta app, not one per client

Each client already gets their own Shopify app, Fly app, Neon project and R2 bucket. The Meta app is the exception: it should be a single NQ-owned app that every client authorises into. Reasons:

- App review and business verification are per-app and take two to four weeks. Repeating that for every client makes onboarding unworkable.
- Meta reviews the app against a demonstrated use case. Five identical apps invite five chances of an inconsistent reviewer decision.
- Merchants authorise a named app on the consent screen. “NQ Shoppable” reads better than “nq-shoppable-umaybanaras”.

The consequence is that `IG_APP_ID` and `IG_APP_SECRET` are the same across all client Fly apps, while the redirect URI differs per client. Every client's callback URL must be listed in the Meta app's valid OAuth redirect URIs.

4.2 Two switches decide who can connect

These are separate settings and both have to be right. Most “it worked for me but not for the client” reports trace back to one of them.

Setting	States	Effect
App mode	Development → Live	A new app starts in Development. It works fully, but only for people holding a role on the app.
Access level (per permission)	Standard → Advanced	`instagram_business_basic` starts at Standard, which again means roles only. App review is what promotes it to Advanced.

Until both are cleared, a merchant can only connect if their Instagram account is listed as a tester *and* they have accepted the invite from inside Instagram. Steps 7 and 8 below cover that.

4.3 Dashboard walkthrough

Steps 1–8 can be done before any code exists. Steps 9–12 need the callback route deployed. Meta reshuffles this dashboard regularly — if a label does not match, look for the nearest equivalent rather than assuming you are in the wrong place.

Before you start

- A Facebook account you are happy to have as the permanent app admin. Use a company account, not a personal one you might lose access to.
- The client's Instagram account must be Business or Creator. Check first — switching takes a minute in the Instagram app under Settings → Account type and tools.
- A decision on which legal entity owns the app. Everything below assumes one NQ-owned app.

Step 1 — Developer account

Go to `developers.facebook.com` and log in. First time only: accept the developer terms and verify by phone or email.

Step 2 — Business portfolio

At `business.facebook.com`, create a business portfolio for NQ if one does not exist. Name it *exactly* as it appears on the registration documents — verification later matches on this string, and a mismatch means a rejection and a resubmission.

Step 3 — Create the app

My Apps → **Create app.**

- App name: **NQ Shoppable**. This is what merchants see on the consent screen.
- Contact email: yours.
- Business portfolio: the one from step 2. Attach it now — retrofitting it later is fiddly.
- Use case: pick the Instagram / account-management option rather than “Other”. Meta tailors the available products to this choice.
- App type: **Business**.

Step 4 — Add the Instagram product

Left sidebar: **Add product** → **Instagram** → **Set up**, then choose **Instagram API with Instagram Login**.

Do not add Facebook Login, and do not pick the Facebook-login variant of the Instagram API. That path requires the merchant's Instagram to be linked to a Facebook Page, which is one more thing to break per client.

Step 5 — Business login settings

Under **Instagram** → **API setup with Instagram login**, open step 3, **Set up Instagram business login** → **Business login settings**. Add one redirect URI per deployment:

```
https://nq-shoppable-lovecovera.fly.dev/auth/instagram/callback
https://nq-shoppable-sumannathwani.fly.dev/auth/instagram/callback
https://nq-shoppable-indethnic.fly.dev/auth/instagram/callback
https://nq-shoppable-queuniverse.fly.dev/auth/instagram/callback
https://nq-shoppable-umaybanaras.fly.dev/auth/instagram/callback
```

Meta matches these exactly — trailing slashes, http versus https, everything. A mismatch fails at the consent screen with a generic error, so paste rather than type.

On the same screen set the two required callbacks:

```
Deauthorize:    https://nq-shoppable-<client>.fly.dev/api/instagram/deauthorize
Data deletion:  https://nq-shoppable-<client>.fly.dev/api/instagram/data-deletion
```

Both are required for review and reviewers do test them. They can point at a single deployment for now.

> **Step 6 — Copy the right credentials**
>
> On that same setup screen you will see an **Instagram app ID** and an **Instagram app secret**. Those are the two you want.
>
> They are *not* the App ID and App Secret shown at the top of the dashboard under **App settings** → **Basic** — those are the Meta app credentials and they will not work for this OAuth flow. This is the single most common way to lose an afternoon here.

Step 7 — Add the client as a tester

App roles → **Roles** → **Add people**, or the Instagram testers section under the Instagram product. Add the client's Instagram username as an **Instagram tester**. While the app is in Development mode with Standard access, only accounts listed here can authorise; everyone else gets a permissions error.

Step 8 — The client accepts the invite

Tell them: open Instagram → **Settings** → **Website permissions** → **Tester invites** (the exact path varies by app version; it sits near “Apps and websites”) → accept the NQ Shoppable invite.

Nothing works until they do this, and there is no signal on our side that it is pending. If a connect attempt fails during a pilot, check this before anything else.

Step 9 — Wire up the secrets

Per client, into `.env.<client>`, then `fly secrets import -a nq-shoppable-<client> < .env.<client>`. See 4.4 for the variable list.

Step 10 — Test the round trip

With the callback route deployed: connect from the settings screen, approve on Instagram, confirm the handle appears and a row lands in `InstagramAccount`. Then list media and check which reels come back missing `media_url`.

Step 11 — Business verification

Business settings → **Security centre** → **Start verification.** Registration certificate, business address, and a phone number or domain Meta can confirm. Start this the same day as step 3 — it is independent of the code and it can bounce on a document mismatch.

Step 12 — App review, then go live

Submit `instagram_business_basic` for Advanced Access, then flip the app from Development to Live. Section 10 covers what the submission must contain. Only after both can a merchant who is not a listed tester connect.

4.4 Environment variables

Added to every client's `.env.<client>` and pushed with `fly secrets import`, alongside the existing eleven:

```
IG_APP_ID=1234567890123456
IG_APP_SECRET=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
IG_REDIRECT_URI=https://nq-shoppable-<client>.fly.dev/auth/instagram/callback
IG_TOKEN_ENC_KEY=<32-byte base64 key, unique per client>
```

`IG_TOKEN_ENC_KEY` encrypts the stored access token at rest. Generate per client with `openssl rand -base64 32`. Unlike the app id and secret this must not be shared between clients — the databases are already isolated and the keys should be too.

> **Prerequisite on the merchant side**
>
> The account being connected must be an Instagram **Business** or **Creator** account. Personal accounts cannot authorise this scope. Most of our clients already run business accounts, but it is worth confirming during onboarding rather than discovering it on the consent screen — switching account type in the Instagram app takes about a minute.

5. OAuth implementation

5.1 Starting the flow

A button on the Instagram settings screen posts to an action that builds the authorize URL and redirects. The `state` parameter carries a signed value containing the shop domain and a nonce — the callback is not an embedded Shopify request, so it has no session of its own and must learn which shop it belongs to from `state`.

```
// app/instagram.server.ts
import crypto from "node:crypto";

const AUTH_BASE  = "https://www.instagram.com/oauth/authorize";
const TOKEN_URL  = "https://api.instagram.com/oauth/access_token";
const GRAPH      = "https://graph.instagram.com";
const SCOPES     = "instagram_business_basic";

export function buildAuthUrl(shop: string): string {
  const nonce = crypto.randomBytes(16).toString("hex");
  const state = signState({ shop, nonce, ts: Date.now() });
  const qs = new URLSearchParams({
    client_id:     process.env.IG_APP_ID!,
    redirect_uri:  process.env.IG_REDIRECT_URI!,
    scope:         SCOPES,
    response_type: "code",
    state,
  });
  return `${AUTH_BASE}?${qs}`;
}

function signState(payload: object): string {
  const body = Buffer.from(JSON.stringify(payload)).toString("base64url");
  const sig  = crypto.createHmac("sha256", process.env.IG_APP_SECRET!)
                     .update(body).digest("base64url");
  return `${body}.${sig}`;
}

export function verifyState(state: string): { shop: string; ts: number } {
  const [body, sig] = state.split(".");
  const expected = crypto.createHmac("sha256", process.env.IG_APP_SECRET!)
                         .update(body).digest("base64url");
  if (!crypto.timingSafeEqual(Buffer.from(sig), Buffer.from(expected))) {
    throw new Error("Bad OAuth state signature");
  }
  const parsed = JSON.parse(Buffer.from(body, "base64url").toString());
  if (Date.now() - parsed.ts > 10 * 60 * 1000) throw new Error("OAuth state expired");
  return parsed;
}
```

5.2 The callback

Two exchanges, back to back. The short-lived token lasts an hour and is useless to us; we immediately trade it for a sixty-day token. Both calls include the app secret, so both must stay server-side.

```
// app/instagram.server.ts
export async function exchangeCode(code: string) {
  const form = new URLSearchParams({
    client_id:     process.env.IG_APP_ID!,
    client_secret: process.env.IG_APP_SECRET!,
```

```
    grant_type:    "authorization_code",
    redirect_uri:  process.env.IG_REDIRECT_URI!,
    code,
  });
  const short = await fetch(TOKEN_URL, { method: "POST", body: form }).then(r => r.json());
  if (short.error_type) throw new Error(short.error_message ?? "Token exchange failed");
  // short = { access_token, user_id, permissions }

  const longUrl = new URL(`${GRAPH}/access_token`);
  longUrl.searchParams.set("grant_type", "ig_exchange_token");
  longUrl.searchParams.set("client_secret", process.env.IG_APP_SECRET!);
  longUrl.searchParams.set("access_token", short.access_token);
  const long = await fetch(longUrl).then(r => r.json());
  if (!long.access_token) throw new Error("Long-lived token exchange failed");

  // Who did we just connect?
  const me = await fetch(
    `${GRAPH}/me?fields=id,username,account_type&access_token=${long.access_token}`
  ).then(r => r.json());

  return {
    igUserId:  String(me.id),
    username:  me.username as string,
    token:     long.access_token as string,
    expiresIn: Number(long.expires_in ?? 5184000),   // seconds, ~60 days
  };
}

// app/routes/auth.instagram.callback.tsx
export const loader = async ({ request }: LoaderFunctionArgs) => {
  const url   = new URL(request.url);
  const code  = url.searchParams.get("code");
  const error = url.searchParams.get("error");
  const state = url.searchParams.get("state");

  if (error || !code || !state) {
    return redirect(`/app/instagram?error=${encodeURIComponent(error ?? "cancelled")}`);
  }

  const { shop } = verifyState(state);
  const account  = await exchangeCode(code);

  await db.instagramAccount.upsert({
    where:  { shop },
    create: {
      shop,
      igUserId:       account.igUserId,
      username:       account.username,
      accessToken:    encryptToken(account.token),
      tokenExpiresAt: new Date(Date.now() + account.expiresIn * 1000),
    },
    update: {
      igUserId:       account.igUserId,
      username:       account.username,
      accessToken:    encryptToken(account.token),
      tokenExpiresAt: new Date(Date.now() + account.expiresIn * 1000),
      revokedAt:      null,
    },
  });

  return redirect(`/app/instagram?connected=${encodeURIComponent(account.username)}`);
};
```

> **The callback route must sit outside the embedded-app auth boundary**
>
> Meta redirects the merchant's browser straight to our Fly host with no Shopify session cookie and no embedded-app context. Calling `authenticate.admin(request)` there will throw or bounce to a login. Put the route under `auth.` alongside the existing `auth.$.tsx`, trust the signed `state` for the shop identity, and redirect back into `/app/instagram` at the end.

5.3 Token lifecycle

Stage	Lifetime	Action
Authorization code	Single use, minutes	Exchanged immediately in the callback.
Short-lived token	1 hour	Never stored. Exchanged for a long-lived token in the same request.
Long-lived token	60 days	Stored encrypted. Refreshable.
Refresh	Resets to 60 days	`GET /refresh_access_token?grant_type=ig_refresh_token`. The token must be at least 24 hours old to refresh.

A token that reaches expiry cannot be refreshed — the merchant has to reconnect. So refresh well ahead of the deadline rather than at it:

```
// app/routes/api.instagram.refresh.tsx  — hit daily by a scheduled Fly machine
export const loader = async ({ request }: LoaderFunctionArgs) => {
  requireCronSecret(request);                    // shared-secret header, not public

  const soon = new Date(Date.now() + 14 * 24 * 3600 * 1000);   // 14-day headroom
  const due  = await db.instagramAccount.findMany({
    where: { revokedAt: null, tokenExpiresAt: { lt: soon } },
  });

  for (const acct of due) {
    try {
      const url = new URL(`${GRAPH}/refresh_access_token`);
      url.searchParams.set("grant_type", "ig_refresh_token");
      url.searchParams.set("access_token", decryptToken(acct.accessToken));
      const res = await fetch(url).then(r => r.json());
      if (!res.access_token) throw new Error(res.error?.message ?? "refresh failed");

      await db.instagramAccount.update({
        where: { shop: acct.shop },
        data: {
          accessToken:    encryptToken(res.access_token),
          tokenExpiresAt: new Date(Date.now() + Number(res.expires_in) * 1000),
          lastError:      null,
        },
      });
    } catch (err) {
      await db.instagramAccount.update({
        where: { shop: acct.shop },
        data:  { lastError: String((err as Error).message).slice(0, 300) },
      });
    }
  }
  return { refreshed: due.length };
};
```

Fourteen days of headroom means a refresh can fail on twelve consecutive days before anyone has to do anything, and the settings screen has time to show a warning banner.

6. Data model

One new model and two new columns on `Video`. Both go in a single forward migration, named so it sorts after the existing ones.

```
/// A merchant's connected Instagram Business/Creator account. One per shop —
/// a Shopify store maps to exactly one Instagram identity in this app.
///
/// accessToken is a long-lived (60-day) Instagram User token, encrypted at rest
/// with IG_TOKEN_ENC_KEY. It is never sent to the browser and never logged.
model InstagramAccount {
  shop            String    @id
  igUserId        String
  username        String
  accessToken     String                    // AES-256-GCM, base64
  tokenExpiresAt  DateTime
  // Cached media listing, so the picker doesn't burn the 200-calls/hour budget
  // on every page view. Refreshed on demand and on a "Refresh" button.
  mediaCache      String?   @db.Text        // JSON array of IgMedia
  mediaCachedAt   DateTime?
  // Set when the merchant disconnects here, or when Meta calls the deauthorize
  // webhook. Row is kept so imported videos retain their provenance.
  revokedAt       DateTime?
  lastError       String?
  connectedAt     DateTime  @default(now())
  updatedAt       DateTime  @updatedAt
}

model Video {
  // ... existing fields unchanged ...

  /// Instagram provenance. Empty for file uploads. Kept so we can show where a
  /// video came from, link back to the original post, and refuse to import the
  /// same reel twice.
  sourceIgShortcode String?
  sourceIgPermalink String?

  @@index([shop, sourceIgShortcode])
}
```

The index is what makes the duplicate check cheap. The existing duplicate check in `app.videos.new.tsx` compares `videoUrl`, which cannot work here — every import produces a fresh R2 key, so the same reel imported twice would look like two different videos. Match on the shortcode instead.

Migration

`prisma/migrations/20260930000000_add_instagram_account/migration.sql`

```
CREATE TABLE "InstagramAccount" (
  "shop"           TEXT PRIMARY KEY,
  "igUserId"       TEXT NOT NULL,
  "username"       TEXT NOT NULL,
  "accessToken"    TEXT NOT NULL,
  "tokenExpiresAt" TIMESTAMP(3) NOT NULL,
  "mediaCache"     TEXT,
  "mediaCachedAt"  TIMESTAMP(3),
  "revokedAt"      TIMESTAMP(3),
  "lastError"      TEXT,
  "connectedAt"    TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt"      TIMESTAMP(3) NOT NULL
);

ALTER TABLE "Video" ADD COLUMN "sourceIgShortcode" TEXT;
ALTER TABLE "Video" ADD COLUMN "sourceIgPermalink" TEXT;

CREATE INDEX "Video_shop_sourceIgShortcode_idx"
  ON "Video"("shop", "sourceIgShortcode");
```

> **Migration ordering**
>
> Prisma applies migration folders in lexicographic order and the deploy is forward-only. The timestamp prefix must sort after `20260822000000_add_video_processing_progress`, which it does. Run `prisma migrate deploy` against `DIRECT_URL`, not the pooled URL — the same trap as every previous migration on this project.

7. Listing and importing reels

7.1 Listing the merchant's media

`/me/media` returns every media item on the account, not just reels, so filter on `media_product_type === "REELS"`. Request `media_url` in the field list even though it may come back absent — its absence is exactly the signal we need to grey out the tile.

```
// app/instagram.server.ts
export type IgMedia = {
  id: string;
  caption?: string;
  media_type: "IMAGE" | "VIDEO" | "CAROUSEL_ALBUM";
  media_product_type?: "FEED" | "REELS" | "STORY" | "AD";
  media_url?: string;          // ABSENT when audio is licensed/copyrighted
  thumbnail_url?: string;
  permalink: string;           // https://www.instagram.com/reel/{shortcode}/
  timestamp: string;
};

const FIELDS = [
  "id", "caption", "media_type", "media_product_type",
  "media_url", "thumbnail_url", "permalink", "timestamp",
].join(",");

export async function listReels(shop: string, opts: { force?: boolean } = {}) {
  const acct = await requireAccount(shop);

  // 200 calls/hour per account. The picker must not call Meta on every render.
  const fresh = acct.mediaCachedAt &&
                Date.now() - acct.mediaCachedAt.getTime() < 15 * 60 * 1000;
  if (fresh && !opts.force && acct.mediaCache) {
    return JSON.parse(acct.mediaCache) as IgMedia[];
  }

  const items: IgMedia[] = [];
  let next: string | null =
    `${GRAPH}/me/media?fields=${FIELDS}&limit=50&access_token=${decryptToken(acct.accessToken)}`;

  // Cap the walk — a merchant with 2000 posts should not cost 40 calls.
  for (let page = 0; next && page < 4; page++) {
    const res: any = await fetch(next).then(r => r.json());
    if (res.error) throw new IgApiError(res.error);
    items.push(...(res.data ?? []));
    next = res.paging?.next ?? null;
  }

  const reels = items.filter(m => m.media_product_type === "REELS" ||
                                  m.media_type === "VIDEO");

  await db.instagramAccount.update({
    where: { shop },
    data:  { mediaCache: JSON.stringify(reels), mediaCachedAt: new Date() },
  });
  return reels;
}
```

7.2 The paste-a-link path

Match, do not fetch. If the shortcode is not in the connected account's media, we refuse and say why.

```
export function extractShortcode(input: string): string | null {
  const m = input.match(
    /instagram\.com\/(?:reels?|p|tv)\/([A-Za-z0-9_-]+)/i
  );
  return m ? m[1] : null;
}

export async function resolveByLink(shop: string, link: string) {
  const shortcode = extractShortcode(link);
  if (!shortcode) {
    return { ok: false as const, reason: "NOT_A_REEL_LINK" };
  }

  let reels = await listReels(shop);
  let hit = reels.find(m => extractShortcode(m.permalink) === shortcode);

  // A reel posted in the last few minutes won't be in a 15-minute-old cache.
  if (!hit) {
    reels = await listReels(shop, { force: true });
    hit = reels.find(m => extractShortcode(m.permalink) === shortcode);
  }

  if (!hit)              return { ok: false as const, reason: "NOT_YOURS" };
  if (!hit.media_url)    return { ok: false as const, reason: "LICENSED_AUDIO", media: hit };
  return { ok: true as const, media: hit };
}
```

`NOT_YOURS` is the security-relevant branch and it is a plain negative result: we never issued a request for the pasted URL, we only failed to find it in a list we were authorised to read. There is no code path in which a link to an account we have not been granted causes any network call to Instagram.

7.3 Importing

Fetch the media fresh rather than trusting the cached `media_url` — those CDN links expire within hours, and the cache can be fifteen minutes old.

```
export async function fetchReelBuffer(shop: string, mediaId: string) {
  const acct = await requireAccount(shop);
  const url  = `${GRAPH}/${mediaId}?fields=${FIELDS}` +
               `&access_token=${decryptToken(acct.accessToken)}`;
  const media: IgMedia = await fetch(url).then(r => r.json());

  if (!media.media_url) {
    throw new ImportError("LICENSED_AUDIO");
  }

  const res = await fetch(media.media_url);
  if (!res.ok) throw new ImportError("CDN_FETCH_FAILED", `${res.status}`);

  const len = Number(res.headers.get("content-length") ?? 0);
  if (len > 300 * 1024 * 1024) throw new ImportError("TOO_LARGE");

  return { media, buffer: Buffer.from(await res.arrayBuffer()) };
}

// app/routes/app.videos.instagram.tsx — action, abridged
const { session, admin } = await authenticate.admin(request);
const mediaId = String(formData.get("mediaId"));

const { media, buffer } = await fetchReelBuffer(session.shop, mediaId);
const shortcode = extractShortcode(media.permalink)!;

// Duplicate check on provenance, not on URL — every import gets a fresh R2 key.
const dupe = await db.video.findFirst({
  where: { shop: session.shop, sourceIgShortcode: shortcode },
});
if (dupe) return { error: "That reel has already been imported." };
```

```
const key      = `videos/${Date.now()}-ig-${shortcode}.mp4`;
const videoUrl = await uploadToR2(key, buffer, "video/mp4");

const video = await db.video.create({
  data: {
    shop: session.shop,
    title: (media.caption ?? "").split("\n")[0].slice(0, 80) || productTitle || "Instagram reel",
    videoUrl,
    streamUrl: null,
    thumbnailUrl: "",
    sourceIgShortcode: shortcode,
    sourceIgPermalink: media.permalink,
    productId, variantId: variantIdNum, productTitle, productUrl,
    productImageUrl,
    price:          price ? parseFloat(price) : null,
    compareAtPrice: compareAtPrice ? parseFloat(compareAtPrice) : null,
    currency: currency || "INR",
  },
});

// From here the flow is byte-identical to a file upload.
processVideo(videoUrl, baseKey, onProgress)
  .then(async (processed) => { /* same .then as app.videos.new.tsx */ })
  .catch(async (err)      => { /* same .catch  */ });

return redirect("/app");
```

> **Factor the post-upload block out rather than copying it**
>
> The `.then` handler in `app.videos.new.tsx` is about eighty lines and carries several hard-won fixes: the throttled progress writes, deleting the raw original only when we uploaded it, and the auto-add that must run after compression because Shopify needs the finished file. Duplicating it into the Instagram route guarantees the two copies drift and one of those fixes gets lost. Extract it into `startVideoProcessing(video, videoUrl, baseKey, { admin, shop, productId })` and call it from both routes.

8. Reels that cannot be imported

The single most important piece of UX in this feature, because it will fire often and because a vague error here will read as a broken product.

In the picker

- Tiles for reels without `media_url` render at reduced opacity with a small lock badge, and are not clickable.
- A tooltip and an inline caption say: *Instagram doesn't allow downloading reels that use licensed audio. Upload the original file instead.*
- A single “Upload file instead” link next to each blocked tile deep-links into the existing upload screen with the caption pre-filled as the title.
- A count at the top of the grid: *18 reels · 7 can't be imported*. Setting the expectation before the merchant clicks is much better than one rejection at a time.

On the paste-a-link path the same condition returns `LICENSED_AUDIO` and shows the same wording, with the file upload offered as the next step.

What not to do

Do not attempt to retrieve the file another way when `media_url` is missing — not via the public page, not via a downloader service, not by asking the merchant for their Instagram password. The field is withheld deliberately, and routing around it converts a Meta policy problem into a copyright problem on a client's storefront.

Note too that a merchant who exports the file from their own phone and uploads it manually has made their own decision about the audio rights on their own content. That is materially different from us fetching it on their behalf, and it is why the manual path stays.

Error taxonomy

Code	Cause	Merchant-facing message
`NOT_A_REEL_LINK`	Regex found no shortcode.	“That doesn't look like an Instagram reel link.”
`NOT_CONNECTED`	No `InstagramAccount` row, or it is revoked.	“Connect your Instagram account first.” with a button.
`NOT_YOURS`	Shortcode absent from `/me/media`.	“That reel isn't on @handle. You can only import reels from the connected account.”
`LICENSED_AUDIO`	`media_url` withheld.	“Instagram doesn't allow downloading this reel because of its audio. Upload the file instead.”
`TOKEN_EXPIRED`	Token past expiry, refresh missed.	“Your Instagram connection expired. Reconnect to keep importing.”
`RATE_LIMITED`	Meta returned code 4 or 17.	“Instagram is rate-limiting us. Try again in a few minutes.”
`CDN_FETCH_FAILED`	`media_url` expired between listing and import.	Retry once automatically with a re-fetched URL, then surface “Couldn't download, try again.”
`TOO_LARGE`	Over 300 MB.	“That reel is too large to import.”
`ALREADY_IMPORTED`	Shortcode already in `Video`.	“That reel has already been imported.” with a link to it.

9. Security and privacy

Concern	Control
Token theft	AES-256-GCM at rest with a per-client `IG_TOKEN_ENC_KEY`. Never returned from a loader, never rendered, never logged. Redact it from error strings before writing `lastError`.
CSRF on the OAuth callback	HMAC-signed `state` containing shop, nonce and timestamp, rejected after ten minutes. Without this, anyone who can hit the callback can bind their own Instagram account to another merchant's shop.
Cross-shop leakage	Every query keys on `session.shop`. A shop can only ever read its own `InstagramAccount` row. Databases are already per-client, so this is defence in depth.
Scope creep	Request `instagram_business_basic` and nothing else. We never need to publish, read messages or read insights.
Merchant revocation	Implement the deauthorize callback. When Meta calls it, set `revokedAt` and clear `accessToken`. Already-imported videos keep working — they live in R2, not on Meta.
Data deletion	Implement the data-deletion callback. On request, delete the account row and, if asked, the imported `Video` rows and their R2 objects via the existing `deleteVideoFromR2`.
Rights record	Store the attestation checkbox result with shop, handle, shortcode, timestamp. Cheap, and useful if a client is ever challenged.
Cron endpoint	`/api/instagram/refresh` requires a shared-secret header. It must not be reachable unauthenticated — it touches every token in the database.

> **Secrets hygiene, same as the rest of the project**
>
> `IG_APP_SECRET` and `IG_TOKEN_ENC_KEY` go into `.env.<client>`, which is already covered by the `.env.*` gitignore and dockerignore rules. Push with `fly secrets import` rather than `fly secrets set` — PowerShell truncates values at `&`, which has already cost us one broken deploy on a Neon URL.

10. Meta app review

The long pole. Start it on day one and build against a test user while it runs.

Requirement	Detail
Business verification	Meta verifies the legal entity behind the app: registration documents, address, a verifiable phone or domain. Allow several days; it can bounce on a document mismatch. Required because our app is used by people who hold no role on it.
Use case description	One paragraph plus a screencast. Say plainly: merchants connect their own Instagram Business account to import their own reels into their own Shopify storefront. Reviewers reject vague descriptions.
Screencast	Must show the whole path: consent screen, connected state, reel picker, an imported reel live on a storefront. Record it against a test store with a test Instagram account.
Privacy policy URL	Must be live and must mention Instagram data specifically: what we store (media ids, permalinks, captions, thumbnails, the access token), why, and for how long.
Deauthorize + data deletion	Both callbacks must be implemented and reachable before submission. Reviewers do check.
Scopes	Only `instagram_business_basic`. Every additional scope is a separate review item and a separate chance of rejection.

Before review passes, the app works fully for Instagram accounts added as testers in the Meta dashboard. That is enough to develop against and enough to demo to a client, but not enough to ship — a merchant who is not a listed tester will see a permission error on the consent screen.

11. Rollout

Phase	Work	Gate
0	Create the Meta app, submit business verification, add one client's Instagram account as a tester.	Day 1. Everything else runs in parallel.
1	Prisma migration, `instagram.server.ts`, OAuth routes, settings screen with connect and disconnect.	Merchant can connect and the handle shows.
2	Extract `startVideoProcessing` from `app.videos.new.tsx`; verify file uploads still work unchanged.	Regression check before anything new depends on it.
3	Reel picker, import action, duplicate check, error taxonomy, licensed-audio states.	A reel imports end to end on the test store.
4	Paste-a-link path on top of the same resolver.	A wrong-account link is rejected without any network call to Instagram.
5	Token refresh endpoint plus scheduled machine; deauthorize and data-deletion callbacks.	Required for submission.
6	Submit for app review with the screencast.	2–4 weeks of waiting, longer if the first submission bounces.
7	Enable per client: add the client's redirect URI, roll out, walk them through connecting.	One client first, then the rest.

> **Ship the fallback first**
>
> Phases 1–5 are worth building regardless, but the manual file upload already works today. If a client needs Instagram content before review clears, the honest answer is: save the reel from the Instagram app and upload the file. It takes them thirty seconds and carries none of the review dependency.

12. Testing checklist

Run every line of this before deploying to a client. The pattern on this project has been that partially-verified deploys cost more than the testing would have.

Connect flow

- Connect succeeds, handle and expiry render on the settings screen.
- Cancelling on the Instagram consent screen returns to the settings screen with a message, not a stack trace.
- A tampered `state` is rejected.
- A `state` older than ten minutes is rejected.
- Connecting a second time replaces the token and clears `revokedAt`.
- Disconnect clears the token; the picker then shows the not-connected state.
- A personal (non-business) Instagram account fails with a readable message.

Listing and import

- The picker shows only the connected account's reels.
- Reels without `media_url` are visibly blocked and not clickable.
- The cache is used on re-render — confirm by counting Meta calls in the logs, not by eye.
- An imported reel lands in R2, compresses, thumbnails and appears in the storefront carousel.
- A reel under five seconds takes the existing short-video path and skips thumbnail generation.
- Importing with a product tagged triggers the Shopify product-media auto-add, as an upload does.
- Importing the same reel twice is refused on the shortcode, not silently duplicated.
- A 200 MB reel imports; a 400 MB one is refused cleanly rather than OOM-ing the machine.

Link path

- A valid own-account link imports.
- A link to any other public account returns `NOT_YOURS` — and the network log shows no request to instagram.com for that URL.
- A link to a private account behaves identically to the above.
- A reel posted two minutes ago imports, exercising the forced cache refresh.
- A non-Instagram URL, a profile URL and a story URL are each rejected with the right code.

Tokens

- The refresh endpoint extends expiry and is rejected without the shared secret.
- An expired token surfaces `TOKEN_EXPIRED` and a reconnect prompt, not a 500.
- The deauthorize callback marks the row revoked; already-imported videos keep playing.
- No token value appears anywhere in `fly logs`, including inside error strings.

13. Open decisions

Question	Recommendation
One Instagram account per shop, or several?	One, for now. Several means a per-video source attribution and a picker that spans accounts — real work for a case no current client has.
Auto-import new reels on a schedule?	Not in v1. It quietly multiplies R2 storage and Shopify video slots without a merchant decision. Revisit once Route B removes the 250-video cap.
Pull the caption in as the video title?	Yes, first line truncated to 80 characters, editable. Better than “Untitled” and merchants can change it.
Import the Instagram thumbnail, or generate our own?	Generate our own with ffmpeg, as today. Keeps one code path and one visual style across uploaded and imported videos.
Show a “from Instagram” badge in the admin list?	Yes — cheap, and it makes provenance visible when a merchant wonders where a video came from.

Sources: Meta for Developers — Business Login for Instagram; Instagram Platform access token reference; IG User Media reference. Verified September 2026. Meta changes this surface often; re-check field names and scope names against the live documentation before implementing.